

[init] Installation et initialisation

DESCRIPTION : SDK Python officiel pour les APIs Africa's Talking
POURQUOI : Envoyer des SMS, recharger en airtime, gérer les appels, les paiements.
CONTEXTE : Applications mobiles africaines, IoT, fintech, e-santé, agritech.

INSTALLATION :
pip install africastalking Version officielle

— INITIALISATION —
import africastalking

africastalking.initialize(username, api\_key) Initialise le SDK
africastalking.initialize("sandbox", "votre\_cle") Mode sandbox (développement)
africastalking.initialize("mon\_app", "cle\_prod") Mode production

— RÈGLES IMPORTANTES —
# username sandbox = TOUJOURS "sandbox" (pas votre vrai username)
# api\_key sandbox = la clé de votre app sandbox sur le dashboard
# Numéros de téléphone : TOUJOURS au format E.164 → "+254711XXXXYY"
# Montants : TOUJOURS avec le code devise → "KES 150", "NGN 500", "UGX 2000"
# Tous les appels sont SYNCHRONES par défaut (bloquants)
# Variante ASYNCHRONE : passer callback=ma\_fonction en paramètre

— ACCÈS AUX SERVICES —
sms = africastalking.SMS Service SMS
airtime = africastalking.Airtime Service Airtime
voice = africastalking.Voice Service Voice (appels)
payment = africastalking.Payment Service Paiements
application = africastalking.Application Infos du compte
token = africastalking.Token Authentification
mobile\_data = africastalking.MobileData Données mobiles
insights = africastalking.Insights SIM Swap / analyses
whatsapp = africastalking.WhatsApp WhatsApp Business (bêta)

— MODE ASYNCHRONE (pour tous les services) —
def on\_finish(error, response): Signature du callback
if error is not None:
print(f"Erreur : {error}")
return
print(response)

# Appel asynchrone (retourne un Thread)
thread = sms.send("Message", ["+254711XXXXYY"], callback=on\_finish)
thread.join() Attendre la fin si nécessaire

EXEMPLES :
import africastalking

africastalking.initialize(username="sandbox", api\_key="atsk...")
sms = africastalking.SMS

res = sms.send("Bonjour depuis AT!", ["+254711XXXXYY"])
print(res)
# {'SMSMessageData': {'Message': 'Sent to 1/1 Total Cost: KES 0.8000',
# 'Recipients': [{'number': '+254711XXXXYY',
# 'status': 'Success', 'cost': 'KES 0.8000',
# 'messageId': 'ATXid...'}]}}

[sms] Service SMS – africastalking.SMS

DESCRIPTION : Envoie, reçoit et gère les SMS via l'API Africa's Talking
POURQUOI : Communication de masse, notifications, OTP, SMS premium.
CONTEXTE : Alertes, marketing, authentification 2FA, services de souscription.

---

— ENVOI DE SMS —

```

sms.send(message, recipients)           Envoi basique
sms.send(
    message="Votre code est 1234",      Texte du message
    recipients=["+254711XXX", "+254733YYY"], Liste de destinataires (format E.164)
    sender_id=None,                     Short code ou ID alphanumérique (optionnel)
    enqueue=False,                      True = livrer sans attendre l'ack telco
    callback=None                       Fonction callback asynchrone (optionnel)
)

# Avec sender_id (short code ou nom alphanumérique enregistré)
sms.send("Msg", ["+254711XXX"], sender_id="AT2FA")
sms.send("Msg", ["+254711XXX"], sender_id="20880")

# Mode enqueue (gros volumes)
sms.send("Msg", destinataires, enqueue=True)

```

---

— SMS PREMIUM —

```

sms.send_premium(
    message="Contenu premium",
    short_code="1234",                  Short code du service premium
    recipients=["+254711XXX"],          Destinataires abonnés
    keyword="MUSIC",                   Mot-clé du service
    link_id=None,                       ID de souscription (optionnel)
    retry_duration_in_hours=None,       Durée de retry (optionnel)
    callback=None
)

```

---

— RÉCUPÉRATION DES SMS ENTRANTS —

```

sms.fetch_messages(last_received_id=0)  Récupère les SMS reçus
sms.fetch_messages()                    Depuis le début
sms.fetch_messages(last_received_id=150) Depuis l'ID 150

```

---

— ABONNEMENTS PREMIUM —

```

sms.fetch_subscriptions(
    short_code="1234",                  Short code du service
    keyword="MUSIC",                   Mot-clé
    last_received_id=0                  Pagination (optionnel)
)

sms.create_subscription(
    short_code="1234",
    keyword="MUSIC",
    phone_number="+254711XXX",          Numéro à abonner
    checkout_token="CkTkn..."        Token de checkout (obligatoire)
)

sms.delete_subscription(
    short_code="1234",
    keyword="MUSIC",
    phone_number="+254711XXX"
)

```

---

— STRUCTURE DE LA RÉPONSE send() —

```

res["SMSMessageData"]["Message"]      Résumé : "Sent to N/M Total Cost: KES X"
res["SMSMessageData"]["Recipients"]    Liste des destinataires
res["SMSMessageData"]["Recipients"][0]["number"]  Numéro
res["SMSMessageData"]["Recipients"][0]["status"]  "Success" ou "InvalidPhoneNumber"...
res["SMSMessageData"]["Recipients"][0]["cost"]    "KES 0.8000"
res["SMSMessageData"]["Recipients"][0]["messageId"] "ATXid_XXXXXXXXXX"

```

---

— STATUTS DE LIVRAISON —

```

"Success"           SMS envoyé avec succès
"InvalidPhoneNumber" Numéro invalide
"NumberNotFound"    Numéro inexistant
"InsufficientCredit" Crédit insuffisant sur le compte
"RiskHold"          Bloqué pour raisons de risque
"UserInBlacklist"   Numéro en liste noire

```

EXEMPLES :

```

sms = africastalking.SMS

# SMS simple
res = sms.send("Alerte capteur !", ["+254711XXXYYY"])

```

```
for r in res["SMSMessageData"]["Recipients"]:
    print(f"{r['number']} → {r['status']} ({r['cost']})")

# SMS groupé avec sender_id
numeros = ["+254711XXX", "+254733YYY", "+254755ZZZ"]
res = sms.send("Bienvenue !", numeros, sender_id="MONAPP")

# SMS asynchrone
def on_sms(error, response):
    if error: print(f"Erreur: {error}")
    else: print(f"Envoyé: {response}")
sms.send("Test async", ["+254711XXX"], callback=on_sms)

# Récupérer les SMS reçus
inbox = sms.fetch_messages()
for msg in inbox.get("SMSMessageData", {}).get("Messages", []):
    print(f"{msg['from']}: {msg['text']}")
```

[air] Service Airtime – africastalking.Airtime

DESCRIPTION : Envoie du crédit téléphonique (airtime) à des numéros de téléphone  
POURQUOI : Récompenser des utilisateurs, payer des agents, fidéliser les clients.  
CONTEXTE : Programmes de fidélité, paiements d'agents, promotions, IoT.

```
FONCTIONS PRINCIPALES :
airtime.send(phone_number, amount)          Envoi à un seul numéro
airtime.send(
    phone_number="+254711XXYYYY",           Format E.164
    amount="KES 50",                         Montant avec code devise
    max_num_retry=0                          Retry en cas d'échec (optionnel)
)

airtime.send(recipients=[...])               Envoi à plusieurs numéros
airtime.send(
    recipients=[
        {"phoneNumber": "+254711XXX", "amount": "KES 50"},
        {"phoneNumber": "+254733YYY", "amount": "NGN 200"},
    ],
    max_num_retry=4                          Retry toutes les 60s pendant 4h
)
```

| STRUCTURE DE LA RÉPONSE            |                               |
|------------------------------------|-------------------------------|
| res["numSent"]                     | Nombre envoyés avec succès    |
| res["totalAmount"]                 | Montant total (string)        |
| res["totalDiscount"]               | Remise totale (string)        |
| res["responses"]                   | Liste des réponses par numéro |
| res["responses"][0]["phoneNumber"] | Numéro destinataire           |
| res["responses"][0]["amount"]      | Montant envoyé                |
| res["responses"][0]["status"]      | "Success" ou message d'erreur |
| res["responses"][0]["requestId"]   | ID de la transaction          |
| res["responses"][0]["discount"]    | Remise appliquée              |

```
EXEMPLES :
airtime = africastalking.Airtime

# Envoi simple
res = airtime.send(phone_number="+254711XXYYYY", amount="KES 50")
print(f"Envoyé: {res['numSent']}, Montant: {res['totalAmount']}")

# Envoi multiple
res = airtime.send(recipients=[
    {"phoneNumber": "+254711XXYYYY", "amount": "KES 100"},
    {"phoneNumber": "+256712345678", "amount": "UGX 5000"},
    {"phoneNumber": "+2348160663047", "amount": "NGN 1500"},
], max_num_retry=3)

for r in res["responses"]:
    print(f"{r['phoneNumber']}: {r['status']} – {r['amount']}")
```

[voice] Service Voice – africastalking.Voice

DESCRIPTION : Passe des appels téléphoniques via l'API Africa's Talking  
POURQUOI : Notifications vocales, IVR, conférences, alertes d'urgence.  
CONTEXTE : Santé, sécurité, finance, services clients, IoT.

FONCTIONS PRINCIPALES :

```
voice.call(
    call_from="+254711082XXX",      Numéro virtuel AT (caller ID enregistré)
    call_to="+254711XXXXYY"        Numéro(s) à appeler
)
voice.call(call_from, call_to)      call_to peut être une liste ou une chaîne CSV

voice.upload_media_file(url, phone_number) Upload fichier audio (MP3/WAV)
# url : URL publique du fichier audio
# phone_number : Numéro virtuel associé au compte

voice.get_num_queued_calls(
    phone_numbers="+254711082XXX",  Numéro virtuel en file d'attente
    queue_name=None                 Nom de la file (optionnel)
)
```

— STRUCTURE DE LA RÉPONSE call() —

```
res = [{"status": "Queued", "phoneNumber": "+254711XXXXYY"}, ...]
res[0]["status"]                  "Queued" = appel initié avec succès
res[0]["phoneNumber"]             Numéro appelé
```

— ACTIONS VOICE XML (callback webhook) —

# Quand le destinataire répond, AT POST vers votre callback URL  
# Vous retournez du XML pour contrôler l'appel :

```
<Response>
  <Say voice="en-US" playBeep="true">Bonjour !</Say>
  <Play url="https://exemple.com/audio.mp3"/>
  <GetDigits timeout="30" finishOnKey="#">
    <Say>Tapez 1 pour français, 2 pour anglais</Say>
  </GetDigits>
  <Dial phoneNumbers="+254733YYY" record="true"/>
  <Record finishOnKey="#" maxLength="60" playBeep="true"
    callbackUrl="https://..." trimSilence="true"/>
  <Enqueue name="SupportQueue" holdMusic="https://.../hold.mp3"/>
  <Dequeue name="SupportQueue" phoneNumber="+254711XXX"/>
  <Conference/>
  <Redirect>https://mon-serveur.com/nouvelle-action</Redirect>
  <Reject/>
</Response>
```

— PARAMÈTRES <Say> —

|                 |                               |
|-----------------|-------------------------------|
| voice="en-US"   | Voix (en-US, en-GB, fr-FR...) |
| playBeep="true" | Joue un bip avant le message  |

— PARAMÈTRES <GetDigits> —

|                           |                                 |
|---------------------------|---------------------------------|
| timeout="30"              | Timeout en secondes             |
| finishOnKey="#"           | Touche de fin de saisie         |
| numDigits="1"             | Nombre de chiffres attendus     |
| callbackUrl="https://..." | URL de callback avec les digits |

— PARAMÈTRES <Dial> —

|                                     |                                     |
|-------------------------------------|-------------------------------------|
| phoneNumbers="+254711X,+254733Y"    | Numéros à appeler (CSV)             |
| ringbackTone="https://.../ring.mp3" | Tonalité pendant la sonnerie        |
| record="true"                       | Enregistre la conversation          |
| sequential="true"                   | Appelle en séquence (pas simultané) |
| callerId="+254711082XXX"            | CallerID à afficher                 |

EXEMPLES :

```
voice = africastalking.Voice

# Passer un appel
res = voice.call(call_from="+254711082XXX", call_to="+254711XXXXYY")
for r in res:
    print(f"{r['phoneNumber']}: {r['status']}")

# Flask callback pour IVR simple
from flask import Flask, request, Response
app = Flask(__name__)

@app.route("/voice/callback", methods=["POST"])
```

```
def voice_callback():
    xml = """<?xml version="1.0"?>
    <Response>
        <GetDigits timeout="30" numDigits="1">
            <Say>Bienvenue. Appuyez sur 1 pour le support, 2 pour la vente.</Say>
        </GetDigits>
    </Response>"""
    return Response(xml, mimetype="text/xml")

@app.route("/voice/digits", methods=["POST"])
def voice_digits():
    digits = request.form.get("dtmfDigits", "")
    if digits == "1":
        xml = "<Response><Say>Vous êtes connecté au support.</Say></Response>"
    else:
        xml = "<Response><Say>Vous êtes connecté aux ventes.</Say></Response>"
    return Response(xml, mimetype="text/xml")
```

|        |                                              |  |
|--------|----------------------------------------------|--|
| [ussd] | Service USSD – africastalking.Ussd / Webhook |  |
|--------|----------------------------------------------|--|

DESCRIPTION : Gère les sessions USSD via un webhook (callback HTTP)  
POURQUOI : Services mobiles sans internet – menus sur téléphone basique.  
CONTEXTE : Banque mobile, santé, agriculture, services de base, zones rurales.

— FONCTIONNEMENT —  
# USSD n'a PAS de méthode Python côté envoi  
# AT envoie un POST HTTP à votre callback URL à chaque interaction utilisateur  
# Vous répondez avec une chaîne préfixée de CON ou END

|                               |                                       |
|-------------------------------|---------------------------------------|
| — PARAMÈTRES REÇUS PAR POST — |                                       |
| request.form["sessionId"]     | Identifiant unique de la session USSD |
| request.form["serviceCode"]   | Code USSD (ex: *384*5#)               |
| request.form["phoneNumber"]   | Numéro de l'utilisateur               |
| request.form["text"]          | Saisies cumulées (séparées par *)     |
| request.form["networkCode"]   | Code réseau opérateur                 |

|                       |                                      |
|-----------------------|--------------------------------------|
| — FORMAT DE RÉPONSE — |                                      |
| "CON " + message      | Session continue (attend une saisie) |
| "END " + message      | Fin de session (ferme le menu)       |

|                          |                                      |
|--------------------------|--------------------------------------|
| — NAVIGATION DANS TEXT — |                                      |
| text == ""               | Premier appel (aucune saisie encore) |
| text == "1"              | L'utilisateur a tapé "1"             |
| text == "1*2"            | L'utilisateur a tapé "1" puis "2"    |
| text == "1*2*123456"     | Plusieurs niveaux de saisie          |
| text.split("**")         | Décompose les saisies en liste       |

```
EXEMPLES :
from flask import Flask, request, Response
import africastalking

app = Flask(__name__)
africastalking.initialize("sandbox", "votre_cle_sandbox")
sms = africastalking.SMS

@app.route("/ussd", methods=["POST"])
def ussd_callback():
    session_id = request.form.get("sessionId")
    phone_number = request.form.get("phoneNumber")
    text = request.form.get("text", "")

    etapes = text.split("**") if text else []

    if text == "":
        # Premier menu
        response = "CON Bienvenue !\n"
        response += "1. Mon numéro\n"
        response += "2. Solde\n"
        response += "3. Aide"

    elif text == "1":
        response = f"END Votre numéro est {phone_number}"
```

```

elif text == "2":
    response = "CON Entrez votre code secret :"

elif len(etapes) == 2 and etapes[0] == "2":
    code = etapes[1]
    if code == "1234":
        response = "END Votre solde est de KES 1,500"
    else:
        response = "END Code incorrect."

elif text == "3":
    response = "END Pour de l'aide, appelez le 0800 XXXXXX"

else:
    response = "END Option invalide."

return Response(response, mimetype="text/plain")

# Django View équivalente
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
def ussd_view(request):
    text = request.POST.get("text", "")
    if text == "":
        response = "CON Choisissez :\n1. Option A\n2. Option B"
    elif text == "1":
        response = "END Vous avez choisi A"
    else:
        response = "END Vous avez choisi B"
    return HttpResponse(response, content_type="text/plain")

```

|                                                |  |
|------------------------------------------------|--|
| [pay] Service Payment – africastalking.Payment |  |
|------------------------------------------------|--|

DESCRIPTION : Effectue des paiements mobiles (M-Pesa, Airtel Money, MTN MoMo...)  
 POURQUOI : Collecter des paiements, payer des agents, transférer de l'argent.  
 CONTEXTE : E-commerce, paiements B2C/B2B, collecte de fonds, paie des employés.

— MOBILE CHECKOUT (collecte un paiement du client) —

```

payment.mobile_checkout(
    product_name="MonProduit",          Nom du produit AT (dashboard)
    phone_number="+254711XXXXYY",       Numéro du payeur
    currency_code="KES",                 Code devise ISO
    amount=565,                          Montant (float)
    provider_channel=None,                Canal fournisseur (optionnel)
    metadata={}                           Métadonnées custom (dict, optionnel)
)
# Codes devise : KES, UGX, TZS, GHS, NGN, ZAR, USD, EUR...
# Retourne {"status": "PendingConfirmation", "description": "...", "transactionId": "..."}

```

— MOBILE B2C (envoi de l'argent au consommateur) —

```

payment.mobile_b2c(
    product_name="MonProduit",
    consumers=[
        {
            "name": "Bob Mwangi",          Nom du bénéficiaire
            "phoneNumber": "+254711XXXXYY", Numéro (E.164)
            "currencyCode": "KES",          Code devise
            "amount": 6766.88,              Montant (float)
            "providerChannel": "1212",      Canal (optionnel)
            "reason": payment.REASON["SalaryPayment"], Raison (optionnel)
            "metadata": {}                  Métadonnées (optionnel)
        }
    ]
)
# payment.REASON keys :
# "SalaryPayment", "BusinessPayment", "PromotionPayment"

```

— MOBILE B2B (paiement entre entreprises) —

```

payment.mobile_b2b(
    product_name="MonProduit",
    business={

```

```

        "provider": "Mpesa",           Fournisseur de paiement
        "destinationChannel": "12345",  Canal de destination
        "destinationAccount": "compte",  Compte destinataire
        "currencyCode": "KES",
        "amount": 1000.00,
        "transferType": payment.TRANSFER_TYPE["BusinessBuyGoods"],
        "metadata": {}
    }
)
# payment.TRANSFER_TYPE keys :
# "BusinessBuyGoods", "BusinessPayBill", "DisburseFundsToBusiness",
# "BusinessToBusinessTransfer"

— BANK CHECKOUT (collecte via virement bancaire) —————
payment.bank_checkout(
    product_name="MonProduit",
    bank_account={
        "accountName": "Compte Courant",
        "accountNumber": "1234567890",
        "bankCode": payment.BANK["FCMB_NG"] Voir constantes ci-dessous
    },
    currency_code="NGN",
    amount=100.00,
    narration="Palement commande #42",
    metadata={}
)

payment.validate_bank_checkout(
    transaction_id="ATPid_...",          ID transaction bank checkout
    otp="123456"                          OTP reçu par le client
)

— BANK TRANSFER (virement bancaire sortant) —————
payment.bank_transfer(
    product_name="MonProduit",
    recipients=[
        {
            "bankAccount": {
                "accountName": "Alice",
                "accountNumber": "1234567890",
                "bankCode": payment.BANK["FCMB_NG"]
            },
            "currencyCode": "NGN",
            "amount": 1500.00,
            "narration": "Salaire",
            "metadata": {}
        }
    ]
)

— CARD CHECKOUT (collecte par carte bancaire) —————
payment.card_checkout(
    product_name="MonProduit",
    currency_code="KES",
    amount=1000.00,
    narration="Achat en ligne",
    payment_card={
        "number": "4111111111111111",      Numéro de carte
        "cvvNumber": 654,                  CVV
        "expiryMonth": 9,
        "expiryYear": 2026,
        "authToken": "tokenSecurise",      Token 3DS
        "countryCode": "NG"               Code pays
    },
    metadata={}
)

payment.validate_card_checkout(
    transaction_id="ATPid_...",
    otp="123456"
)

— OPÉRATIONS DE PORTEFEUILLE —————
payment.wallet_transfer(
    product_name="MonProduit",

```

```
target_product_code=1234,          Code du produit cible
currency_code="KES",
amount=5000.00,
metadata={}
)

payment.topup_stash(
    product_name="MonProduit",
    currency_code="KES",
    amount=1000.00,
    metadata={}
)

— REQUÊTES DE STATUT —
payment.find_transaction(transaction_id="ATPid_...") Statut d'une transaction
payment.fetch_wallet_balance() Solde du portefeuille

— CONSTANTES BANQUES —
payment.BANK["FCMB_NG"]           First City Monument Bank (Nigéria)
payment.BANK["ZENITH_NG"]         Zenith Bank (Nigéria)
payment.BANK["ACCESS_NG"]        Access Bank (Nigéria)
payment.BANK["GTBANK_NG"]        Guaranty Trust Bank (Nigéria)
payment.BANK["ECOBANK_NG"]       Ecobank Nigéria
payment.BANK["DIAMOND_NG"]       Diamond Bank (Nigéria)
payment.BANK["PROVIDUS_NG"]      Providus Bank
payment.BANK["UNITY_NG"]         Unity Bank
payment.BANK["STANBIC_NG"]       Stanbic IBTC Bank
payment.BANK["STERLING_NG"]     Sterling Bank
payment.BANK["PARKWAY_NG"]      Parkway Bank
payment.BANK["AFRIBANK_NG"]     Afribank
payment.BANK["ENTERPRISE_NG"]    Enterprise Bank
payment.BANK["FIDELITY_NG"]     Fidelity Bank
payment.BANK["HERITAGE_NG"]     Heritage Bank
payment.BANK["KEYSTONE_NG"]     Keystone Bank
payment.BANK["SKYE_NG"]         Skye Bank
payment.BANK["STANCHART_NG"]    Standard Chartered
payment.BANK["UNION_NG"]        Union Bank
payment.BANK["UBA_NG"]          United Bank for Africa
payment.BANK["WEMA_NG"]         Wema Bank
```

```
EXEMPLES :
payment = africastalking.Payment

# Collecter un paiement M-Pesa
res = payment.mobile_checkout(
    product_name="MonApp",
    phone_number="+254711XXXXYY",
    currency_code="KES",
    amount=500,
    metadata={"orderId": "ORD-001"}
)
print(res["status"]) # "PendingConfirmation"

# Payer un employé (B2C)
res = payment.mobile_b2c(
    product_name="Paie",
    consumers=[{
        "name": "Alice Njeri",
        "phoneNumber": "+254711XXXXYY",
        "currencyCode": "KES",
        "amount": 25000.00,
        "reason": payment.REASON["SalaryPayment"]
    }]
)
)
```

|                                                       |  |
|-------------------------------------------------------|--|
| [data] Service MobileData – africastalking.MobileData |  |
|-------------------------------------------------------|--|

DESCRIPTION : Envoie des forfaits data mobile à des clients  
POURQUOI : Offrir de la data à des utilisateurs, des agents ou des clients.  
CONTEXTE : Programmes de fidélité, accès à des services numériques, IoT.

FONCTIONS PRINCIPALES :  
mobile\_data.send(



```

product_name="MonProduit",          Produit AT Mobile Data
recipients=[
    {
        "phoneNumber": "+254711XXXXYY", Numéro bénéficiaire (E.164)
        "quantity": 10,               Quantité de data (MB ou unités)
        "unit": "MB",                 Unité : "MB" ou "GB"
        "validity": "Day",            Validité : "Day", "Week", "Month", "BiMonth", "Quarterly", "HalfYear", "Year", "HalfMonth"
        "isPromotion": False          Promotion (bool, optionnel)
    }
]
)

```

```

mobile_data.find_transaction(transaction_id) Statut d'une transaction data
mobile_data.fetch_wallet_balance()           Solde du portefeuille data

```

#### EXEMPLES :

```
mobile_data = africastalking.MobileData
```

```

res = mobile_data.send(
    product_name="DataPromo",
    recipients=[{
        "phoneNumber": "+254711XXXXYY",
        "quantity": 500,
        "unit": "MB",
        "validity": "Day",
    }]
)
print(res)

```

|                                                |
|------------------------------------------------|
| [app] Application – africastalking.Application |
|------------------------------------------------|

DESCRIPTION : Récupère les informations du compte Africa's Talking  
 POURQUOI : Vérifier le solde du compte, surveiller les dépenses.  
 CONTEXTE : Monitoring, alertes de solde bas, tableaux de bord.

#### FONCTIONS PRINCIPALES :

```
application.fetch_application_data()      Infos du compte (solde, username...)
```

#### — STRUCTURE DE LA RÉPONSE —

```
res["UserData"]["balance"]              Solde du compte (str, ex: "KES 100.50")
```

#### EXEMPLES :

```
application = africastalking.Application
```

```

res = application.fetch_application_data()
print(f"Solde : {res['UserData']['balance']}")

```

|                                    |
|------------------------------------|
| [tok] Token – africastalking.Token |
|------------------------------------|

DESCRIPTION : Génère des tokens d'authentification alternatifs à la clé API  
 POURQUOI : Sécuriser les applications côté client sans exposer la clé API.  
 CONTEXTE : Applications web, mobile, intégrations tierces.

#### FONCTIONS PRINCIPALES :

```

token.generate_auth_token()             Génère un token d'auth temporaire
token.create_checkout_token(phone_number) Token pour créer un abonnement premium

```

#### — STRUCTURE DE LA RÉPONSE generate\_auth\_token() —

```

res["token"]                            Token JWT temporaire (str)
res["lifetimeInSeconds"]                 Durée de vie du token (int)

```

#### — STRUCTURE DE LA RÉPONSE create\_checkout\_token() —

```

res["token"]                            Token de checkout (str, "CkTkn...")
res["description"]                       Description du statut

```

#### EXEMPLES :

```
token = africastalking.Token
```

```

# Token d'auth (pour remplacer l'API key)
res = token.generate_auth_token()
print(f"Token: {res['token']} (valable {res['lifetimeInSeconds']}s)")

```

```
# Token de checkout pour créer un abonnement premium
checkout = token.create_checkout_token("+254711XXXXYY")
checkout_token = checkout["token"]
sms.create_subscription("1234", "MUSIC", "+254711XXXXYY", checkout_token)
```

[ins] Insights – africastalking.Insights

DESCRIPTION : Analyse les numéros de téléphone (SIM Swap, informations réseau)  
POURQUOI : Détecter les fraudes liées aux échanges de SIM.  
CONTEXTE : Banques, fintech, sécurité des comptes, authentification forte.

FONCTIONS PRINCIPALES :  
insights.check\_sim\_swap\_state(phone\_numbers) Vérifie l'état de swap SIM  
insights.check\_sim\_swap\_state(["+254711XXX", "+254733YYY"])

— STRUCTURE DE LA RÉPONSE —

|                                  |                                  |
|----------------------------------|----------------------------------|
| res                              | Dict avec numéros comme clés     |
| res["+254711XXXXYY"]["swapped"]  | Bool – True si SIM a été swappée |
| res["+254711XXXXYY"]["swapTime"] | Timestamp du dernier swap        |

EXEMPLES :  
insights = africastalking.Insights  
  
res = insights.check\_sim\_swap\_state(["+254711XXXXYY"])  
for numero, info in res.items():  
 if info.get("swapped"):  
 print(f"⚠ {numero} : SIM swappée ! Bloquer la transaction.")

[wa] WhatsApp – africastalking.WhatsApp

DESCRIPTION : Envoie des messages WhatsApp Business  
POURQUOI : Communiquer via WhatsApp avec les clients.  
CONTEXTE : Service client, notifications transactionnelles, alertes.

FONCTIONS PRINCIPALES :  
whatsapp.send(  
 body={  
 "message": "Bonjour !",  
 # ou  
 "mediaUrl": "https://.../img.jpg",  
 "mediaType": "image",  
 "caption": "Description"  
 },  
 wa\_number="+254711082XXX",  
 phone\_number="+254711XXXXYY"  
)

|                                       |
|---------------------------------------|
| Message texte                         |
| URL d'image/document                  |
| "image", "document", "audio", "video" |
| Légende (optionnel)                   |
| Numéro WA associé à votre compte      |
| Destinataire                          |

EXEMPLES :  
whatsapp = africastalking.WhatsApp  
  
# Message texte  
res = whatsapp.send(  
 body={"message": "Votre commande est confirmée !"},  
 wa\_number="+254711082XXX",  
 phone\_number="+254711XXXXYY"  
)  
  
# Message avec image  
res = whatsapp.send(  
 body={"mediaUrl": "https://exemple.com/facture.pdf",  
 "mediaType": "document",  
 "caption": "Votre facture du mois"},  
 wa\_number="+254711082XXX",  
 phone\_number="+254711XXXXYY"  
)

[cb] Callbacks et webhooks – notifications entrantes

DESCRIPTION : Traite les notifications envoyées par AT vers votre serveur  
POURQUOI : Réagir aux SMS entrants, rapports de livraison, paiements reçus.  
CONTEXTE : Serveurs Flask, Django, FastAPI recevant les POST de AT.

#### — SMS ENTRANT (Incoming Message Callback) —

# Champs POST envoyés par AT :

|                             |                                  |
|-----------------------------|----------------------------------|
| request.form["id"]          | ID du message                    |
| request.form["from"]        | Numéro expéditeur                |
| request.form["to"]          | Short code/numéro destinataire   |
| request.form["text"]        | Contenu du SMS                   |
| request.form["date"]        | Date de réception (ISO)          |
| request.form["linkId"]      | ID de souscription (SMS premium) |
| request.form["networkCode"] | Code réseau opérateur            |

#### — RAPPORT DE LIVRAISON (Delivery Report Callback) —

|                               |                                |
|-------------------------------|--------------------------------|
| request.form["id"]            | ID du message                  |
| request.form["status"]        | Statut de livraison            |
| request.form["phoneNumber"]   | Numéro destinataire            |
| request.form["networkCode"]   | Code réseau                    |
| request.form["failureReason"] | Raison d'échec (si applicable) |
| request.form["retailer"]      | Détails opérateur              |

# Statuts possibles : Success, Sent, Failed, Rejected, Buffered,  
# RiskHold, InvalidRecipientAddress, ForbiddenCountryCode...

#### — PAIEMENT REÇU (Payment Notification Callback) —

|                                 |                                    |
|---------------------------------|------------------------------------|
| request.form["transactionId"]   | ID unique de la transaction        |
| request.form["category"]        | "MobileCheckout", "B2B", "B2C"...  |
| request.form["provider"]        | "Mpesa", "Airtel", "MTN"...        |
| request.form["providerChannel"] | Canal fournisseur                  |
| request.form["clientAccount"]   | Compte client                      |
| request.form["productName"]     | Nom du produit AT                  |
| request.form["sourceType"]      | Source du paiement                 |
| request.form["source"]          | Numéro source                      |
| request.form["destinationType"] | Type de destination                |
| request.form["destination"]     | Numéro destination                 |
| request.form["value"]           | Montant avec devise ("KES 150.00") |
| request.form["transactionFee"]  | Frais de transaction               |
| request.form["providerRefId"]   | Référence fournisseur              |
| request.form["status"]          | "Success", "Failed"                |
| request.form["description"]     | Description du statut              |
| request.form["requestMetadata"] | Métadonnées JSON (si fourni)       |

#### — EXEMPLE FLASK COMPLET —

```
from flask import Flask, request, jsonify
import africastalking, json

app = Flask(__name__)
africastalking.initialize("sandbox", "votre_cle")
sms = africastalking.SMS

@app.route("/sms/incoming", methods=["POST"])
def sms_incoming():
    expéditeur = request.form.get("from")
    message = request.form.get("text", "").strip().upper()
    print(f"SMS de {expéditeur}: {message}")
    if message == "BALANCE":
        sms.send("Votre solde est de KES 500", [expéditeur])
    elif message == "HELP":
        sms.send("Envoyez BALANCE, HELP ou STOP", [expéditeur])
    return jsonify({"status": "OK"}), 200

@app.route("/sms/delivery", methods=["POST"])
def sms_delivery():
    msg_id = request.form.get("id")
    statut = request.form.get("status")
    numero = request.form.get("phoneNumber")
    print(f"Livraison {msg_id} → {numero}: {statut}")
    return jsonify({"status": "OK"}), 200

@app.route("/payment/callback", methods=["POST"])
def payment_callback():
    txn_id = request.form.get("transactionId")
    statut = request.form.get("status")
    montant = request.form.get("value")
    print(f" Paiement {txn_id}: {statut} – {montant}")
```

```

return jsonify({"status": "OK"}), 200

if __name__ == "__main__":
    app.run(debug=True, port=8080)

```

## RÉCAPITULATIF DES IMPORTS ET SERVICES

### — INSTALLATION —

```
pip install africastalking
```

### — INITIALISATION —

```

import africastalking
africastalking.initialize("sandbox", "votre_api_key") # Sandbox
africastalking.initialize("mon_app", "votre_api_key") # Production

```

### — SERVICES —

```

sms          = africastalking.SMS          send(), send_premium(), fetch_messages()
airtime      = africastalking.Airtime      send()
voice        = africastalking.Voice        call(), upload_media_file()
payment      = africastalking.Payment      mobile_checkout(), mobile_b2c(), mobile_b2b()
application  = africastalking.Application  fetch_application_data()
token        = africastalking.Token        generate_auth_token(), create_checkout_token()
mobile_data  = africastalking.MobileData   send(), find_transaction()
insights     = africastalking.Insights     check_sim_swap_state()
whatsapp     = africastalking.WhatsApp     send()

```

### — GUIDE DE DÉCISION RAPIDE —

|                            |                                                  |
|----------------------------|--------------------------------------------------|
| Envoyer un SMS             | → sms.send(message, [numeros])                   |
| SMS avec expéditeur nommé  | → sms.send(msg, nums, sender_id="MONAPP")        |
| SMS en masse (enqueue)     | → sms.send(msg, nums, enqueue=True)              |
| Recharger en airtime       | → airtime.send(phone_number, "KES 50")           |
| Appel sortant              | → voice.call(call_from="+...", call_to="+...")   |
| IVR / Menu vocal           | → Webhook Flask → réponse XML                    |
| Menu USSD                  | → Webhook Flask → réponse "CON ..." ou "END ..." |
| Collecter un paiement      | → payment.mobile_checkout(...)                   |
| Payer un employé           | → payment.mobile_b2c(...)                        |
| Envoyer des données mobile | → mobile_data.send(...)                          |
| Solde du compte            | → application.fetch_application_data()           |
| Détecter un SIM Swap       | → insights.check_sim_swap_state([numero])        |
| Message WhatsApp           | → whatsapp.send(body, wa_number, phone_number)   |
| Environnement de test      | → username="sandbox", api_key=clé_sandbox        |